

Chapter 24 - Executable Knowledge: Stage 7 of Semantic Knowledge Development Lifecycle

From Intelligent Knowledge to Actionable Systems

The first six stages of the Semantic Knowledge Development Lifecycle (SKDL) transformed an abstract conceptual vocabulary into a **validated, intelligent, machine-reasonable** knowledge asset.

- Conceptual Modeling gave us structure.
- Semantic Description gave us meaning.
- Knowledge Reuse gave us composability.
- Semantic Governance gave us discipline.
- Semantic Validation gave us trust.
- Semantic Reasoning gave us intelligence.

Now, as knowledge begins to flow from ontology models into **production systems**, a new challenge emerges:

"How do we act on this knowledge?"

Executable Knowledge answers this question.

It is the discipline of encoding semantic knowledge with sufficient logical constraints to be interpreted deterministically and to drive action -- closing the loop between reasoning and execution.

Executable Knowledge transforms an ontology from a system that *knows* into a system that *acts*.

This chapter examines Exercise 22 through 26 from Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial* through the **lens of executability**. While **Exercise 20** and **Exercise 21 (Chapter (23))** demonstrated the power of reasoning, **Exercise 22-26** introduce the OWL constructs that make reasoning *actionable*: universal restrictions, closure axioms, enumerated classes, `hasValue` restriction, and cardinality restrictions.

These constructs are the **execution primitives** of an ontology. They are what separate a descriptive model from an operational one.

Along the way, we will examine executability from multiple complementary perspectives: mathematical, engineering, architectural, and industrial -- revealing that executable knowledge is not a new invention, but the natural culmination of the entire SKDL arc.

Table of Content

- [24.1 Chapter Introduction -- From Intelligence to Action](#)
 - [24.1.1 The Journey So Far](#)
 - [24.1.2 The New Question](#)
 - [24.1.3 The Central Insight](#)
- [24.2 Learning Objectives](#)
- [24.3 Positioning Within SKDL -- Stage 7 Highlighted](#)
- [24.4 What is Executable Knowledge?](#)
 - [24.4.1 A Working Definition](#)
 - [24.4.2 The Three Layers of Semantic Knowledge](#)
 - [24.4.3 Executable Knowledge vs. Reasoning vs. Validation](#)
 - [24.4.4 The Open World Assumption and Executability](#)
- [24.5 Exercise 22 -- Universal Restrictions and Deterministic Classification](#)
 - [24.5.1 The Engineering Objective](#)
 - [24.5.2 Implementation: Creating the Universal Restriction](#)
 - [24.5.3 The Result: What the Reasoner Does Not Do](#)
 - [24.5.4 Why the Universal Restriction Is Not Enough](#)
 - [24.5.5 Engineering Perspective: Exercise 22 as a Case-Study](#)
- [24.6 Exercise 23 -- Closure Axioms and the Open World Assumption](#)
 - [24.6.1 The Engineering Objective](#)
 - [24.6.2 Implementation: Adding Closure Axioms to `MargheritaPizza` and `SohoPizza`](#)

- [24.6.3 Extending the Demonstration: Closure at the Individual Level](#)
- [24.6.4 From Description to Specification](#)
- [24.6.5 Why Closure Axioms Matter for Execution](#)
- [24.6.6 Engineering Perspective: Exercise 23 as a Case Study](#)
- [24.7 A Brief Note on OWL Design Patterns](#)
- [24.8 Exercise 24 -- Enumerated Classes and Deterministic Value Sets](#)
 - [24.8.1 The Engineering Objective](#)
 - [24.8.2 Implementation: Creating the Enumerated Class](#)
 - [24.8.3 The Result: A Deterministic Value Domain](#)
 - [24.8.4 Why Enumerated Classes Matter for Execution](#)
 - [24.8.5 Engineering Perspective: Exercise 24 as a Case Study](#)

24.1 Chapter Introduction -- From Intelligence to Action

24.1.1 The Journey So Far

The first six stages of the Semantic Knowledge Development Lifecycle established the foundations of ontology engineering.

Stage 1 -- Conceptual Modeling identified the concepts that define a knowledge domain and organized them into a coherent semantic taxonomy.

Stage 2 -- Semantic Description enriched those concepts with formal meaning using Description Logic, OWL restrictions, and class expressions.

Stage 3 -- Knowledge Reuse transformed validated semantic knowledge into reusable assets that could be shared, extended, and specialized systematically.

Stage 4 -- Semantic Governance established the policies, standards, and processes required to maintain semantic integrity across an evolving knowledge ecosystem.

Stage 5 -- Semantic Validation verified that the ontology is logically consistent, semantically complete, and fit for purpose before deployment.

Stage 6 -- Semantic Reasoning derived new knowledge through automated inference -- computing the logical consequences of the asserted axioms.

By the end of **Stage 6**, the `Pizza` ontology had become intelligent. The reasoner auto-classified `MargheritaPizza` and other cheese-topped pizzas as subclasses of `CheesyPizza`. The inferred hierarchy now contained knowledge that was never explicitly asserted.

The ontology had become **intelligent**.

But intelligent is **not action**.

An intelligent ontology that cannot act is like a brilliant advisor who never speaks. It contains valuable knowledge, but it does not use that knowledge to change the world.

It is passive, not active!

This is where **Stage 7 -- Executable Knowledge** enters the lifecycle.

24.1.2 The New Question

Reasoning asked: ***What else can this model tell us?***

Executable Knowledge asks: ***What should happen, given what we know?***

The distinction is fundamental.

Reasoning derives new knowledge from a validated ontology. Executable Knowledge uses that derived knowledge to **drive action**.

Without reasoning, execution has no foundation.

Without execution, reasoning has no consequence.

Together, they produce actionable intelligence.

24.1.3 The Central Insight

The central insight of this chapter is deceptively simple:

Every OWL axiom you write is a potential execution rule. The question is not whether your ontology can be executed -- it is whether you have written the axioms that make execution possible.

The reasoner infers facts.

But inference alone does not trigger action.

Action requires:

- **Determinism** -- the ontology must produce a single, unambiguous result
- **Completeness** -- the ontology must guarantee that required data is present
- **Closure** -- the ontology must define boundaries, not leave them open

These are not automatic. They must be **engineered** through deliberate axioms.

Exercise 22-26 introduce these axioms.

24.2 Learning Objectives

After completing this chapter, you should be able to achieve the following learning outcomes:

Knowledge:

- Explain why Executable Knowledge is the seventh stage of the Semantic Knowledge Development Lifecycle (SKDL)
- Distinguish between descriptive knowledge and executable knowledge
- Understand the role of universal restrictions, closure axioms, enumerated classes, `hasValue` restrictions, and cardinality restrictions in enabling execution
- Explain how the Open World Assumption affects executability
- Understand how OWL constructs become execution primitives in the EKA framework

Practical Skills:

- Create universal restrictions (`only`) in Protégé
- Define closure axioms using combinations of `only` and cardinality restrictions
- Create enumerated classes using `EquivalentTo`
- Define `hasValue` restrictions to bind properties to specific individuals
- Define cardinality restrictions (`min` , `max` , `exactly`) to enforce data completeness
- Test executability by running the reasoner against test individuals

Mathematical Perspective:

- Understand how closure axioms convert open-world semantics to closed-world specifications
- Explain the mathematical relationships between `some` , `only` , `exactly` , and `value` restrictions
- Understand how enumeration defines a finite value domain
- Explain how cardinality restrictions constrain the extension of a class

Historical Perspective:

- Trace the concept of executability from early expert systems to modern rule engines
- Understand how the "knowledge is action" principle connects to cybernetics, control theory, and business process management

Engineering Perspective:

- Understand execution as the operationalization of semantic knowledge
- Explain why closure axioms are necessary for deterministic execution
- Map executable knowledge to the Θ (Trigger) and Φ (Execution) layers in the EKA framework
- Apply executable knowledge to industrial scenarios (manufacturing, compliance, safety)

24.3 Positioning Within SKDL -- Stage 7 Highlighted

As introduced in **Chapter (17)**, ontology engineering progresses through seven stages of the Semantic Knowledge Development Lifecycle (SKDL), each stage contributing a distinct engineering objective toward the construction of **executable semantic knowledge systems**.

Having completed **Stage 6 -- Semantic Reasoning** in the previous chapter, this chapter advances to **Stage 7 -- Executable Knowledge**.

The objective of this stage is to operationalize semantic knowledge -- to encode the ontology with the constraints required for **deterministic, actionable** execution.

At the completion of **Stage 6**, the `Pizza` ontology contained an inferred hierarchy that included knowledge derived by the reasoner. However, the ontology was still **descriptive**. It could tell us that `MargheritaPizza` is a subclass of `CheesyPizza` . But it could not yet tell us what to **do** about it.

Stage 7 addresses this gap by introducing:

- **Universal Restrictions** -- encoding exclusion and deterministic classification
- **Closure Axioms** -- converting open-world descriptions to closed-world specifications
- **Enumerated Classes** -- defining finite value domains
- **hasValue Restrictions** -- binding properties to specific values
- **Cardinality Restrictions** -- enforcing data completeness

Rather than introducing new semantic constructs, this stage establishes the engineering discipline required to make semantic knowledge actionable.

The primary deliverable of **Stage 7** is therefore:

not a new logical axiom, but an executable ontology -- one whose inferred knowledge can drive triggers, execute actions, and produce real-world effects.

Within the EKA framework, **Stage 7** establishes the Θ -- **Trigger** and Φ -- **Execution** layers by providing the semantic conditions that fire triggers and the action bindings that execute them.

The progression across all sever stages can now be seen in full:

SKDL Stage	Primary Outcome	EKA Contribution
Stage 1 -- Conceptual Modeling	Identify concepts and organize them into a coherent semantic taxonomy	K (Knowledge Graph) (Structure)
Stage 2 -- Semantic Description	Enrich concepts with formal semantic meaning through Description Logic and OWL restrictions	K (Meaning) + R (Readiness)
Stage 3 -- Knowledge Reuse	Transform validated semantic knowledge into reusable assets	K (Composable) + Γ (Prepared)
Stage 4 -- Semantic Governance	Maintain semantic integrity through policies, standards, and lifecycle management	Γ (Governance)
Stage 5 -- Semantic Validation	Verify correctness before deployment through consistency checking and constraint validation	Γ + K (Validation)
Stage 6 -- Semantic Reasoning	Derive new knowledge through automated inference	R (Reasoning & Rules)
Stage 7 -- Executable Knowledge	Operationalize semantic knowledge for deterministic action	Θ (Trigger) + Φ (Execution)

Each stage has built upon the previous one.

- **Stage 1** gave us concepts.
- **Stage 2** gave us meaning.
- **Stage 3** gave us reusability.
- **Stage 4** gave us governance.
- **Stage 5** gave us trust.

- **Stage 6** gave us intelligence.
- **Stage 7** now gives us **action**.

24.4 What is Executable Knowledge?

24.4.1 A Working Definition

Executable Knowledge is semantic knowledge that has been encoded with sufficient logical constraints to be interpreted deterministically and to drive action.

It answers the question:

"What should happen, given what we know?"

Not "what is true?" (that is reasoning).

Not "what is correct?" (that is validation).

But "what should be **done**?"

This is fundamentally an **operationalization** activity. It is the moment when the ontology becomes a decision system -- when its axioms do not merely describe the world but specify what to do about it.

Key characteristics of executable knowledge:

- **Deterministic** -- Given the same facts, the same action is triggered. No ambiguity.
- **Closed** -- The ontology specifies boundaries, not open-ended descriptions.
- **Complete** -- The ontology guarantees that required data is present before acting.
- **Actionable** -- The ontology binds semantic conditions to executable actions.

Key Insight:

"Reasoning tells us what is true. Executable Knowledge tells us what to do about it. Without executability, the ontology is a library. With it, the ontology is a decision system."

24.4.2 The Three Layers of Semantic Knowledge

Semantic knowledge in an ontology operates at three distinct layers:

Layer	Question	OWL Constructs	Stage
Description	What is true?	SubClassOf , some , only	Stage 1-2
Validation	What is trustworthy?	Disjointness, domain/range, SHACL	Stage 4-5
Execution	What should happen?	Closure axioms, cardinality, hasValue , enumerated classes	Stage 7

The **description layer** defines concepts. The **validation layer** ensures correctness. The **execution layer** enables action.

Without the execution layer, the ontology is correct but passive. With it, the ontology is actionable.

Key Insight:

"Description without validation is unchecked. Validation without execution is theoretical. Execution without description and validation is dangerous. The three layers together form a complete semantic system."

24.4.3 Executable Knowledge vs. Reasoning vs. Validation

It is important to distinguish executability from reasoning and validation:

Activity	Question	Timing	Output
Validation	"Is this model correct?"	Before reasoning	Pass/fail -- the ontology is consistent or not
Reasoning	"What else can this model tell us?"	After validation	Inferred facts -- new classification, relationships
Executable Knowledge	"What should happen?"	After reasoning	Actions -- triggered operations, workflow steps

Validation ensures correctness. **Reasoning** derives new knowledge. **Executable Knowledge** acts on that knowledge.

The key distinction is this: **reasoning produces facts; executability produces actions.**

A reasoned ontology knows that a patient is `HighRiskPatient`, then an executable ontology sends an alert to the clinician.

Key Insight:

"Reasoning is the brain. Execution is the hand. A brain without a hand is a philosopher. A hand without a brain is a hazard. Together, they produce intelligent action."

24.4.4 The Open World Assumption and Executability

The Open World Assumption (OWA) is foundational to OWL. It states that just because something is not asserted does not mean it is false. This is correct for reasoning -- but it creates a problem for execution.

Consider a pizza with `hasTopping some MozzarellaTopping`. Under OWA, the reasoner cannot conclude that the pizza has **only** mozzarella. There might be other toppings we simply don't know about.

For reasoning, this is correct.

For execution, it is problematic.

An execution system that recommends a pizza based on an incomplete description may recommend the wrong thing.

Key Insight:

"The Open World Assumption is correct for reasoning and dangerous for execution. Execution requires the closed world. **Closure axioms** are the bridge."

This tension is the reason **Exercise 22-26** exist. They introduce the OWL constructs that allow us to **close** the world when execution requires it.

24.5 Exercise 22 -- Universal Restrictions and Deterministic Classification

Having established the principles of executable knowledge, we now turn to its practical implementation through **Exercise 22** in Michael DeBellis' *Protégé 5 New OWL Pizza Tutorial*.

This exercise represents the first hands-on activity of **Stage 7 -- Executable Knowledge** within the Semantic Knowledge Development Lifecycle (SKDL).

Unlike previous exercises, which introduced individual OWL constructs or semantic modeling patterns, **Exercise 22** focuses on the construct that makes deterministic classification possible: **the universal restriction**.

24.5.1 The Engineering Objective

Exercise 22 introduces that **universal restriction**, represented in OWL Manchester Syntax by the keyword `only`.

The universal restriction specifies that **all** values of a property must satisfy a specified condition. It does not require that any values exist -- only that, if they exist, they must satisfy the condition.

The definition for a vegetarian pizza, for example, might be:

```

Class: VegetarianPizza
  SubClassOf: Pizza
  SubClassOf: hasTopping only (CheeseTopping or VegetableTopping)

```

This means: "Every topping on a vegetarian pizza must be either a cheese topping or a vegetable topping."

The engineering objective of **Exercise 22** is to introduce the `only` construct and demonstrate how it enables deterministic classification -- the ability to **execute** as well as include.

Key Insight:

- Existential restrictions tell the reasoner what must be present.
- Universal restrictions tell the reasoner what must not be present.

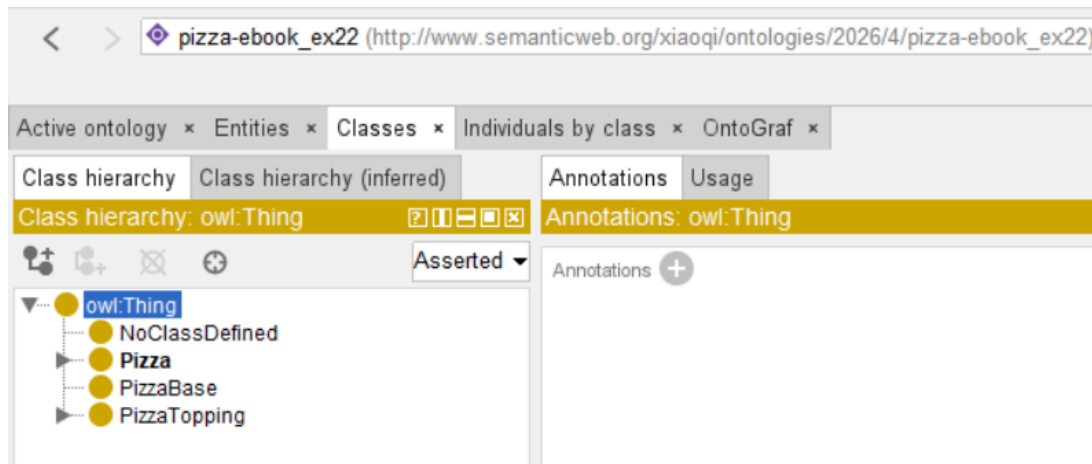
Execution needs both.

24.5.2 Implementation: Creating the Universal Restriction

Let's walk through the implementation step by step. As you'll see, the exercise **deliberately** includes a moment where the expected result does **not** appear -- and that moment is the most important part of the lesson.

Step 1: Open the `Pizza` ontology

The RDF file for this exercise is named as `pizza-ebook_ex22.rdf`. You may find it in the ebook's `\rdf` folder. Open it in Protégé. Navigate to the Classes tab. (Tip: you may open `pizza-ebook_ex21.rdf` as the starting ontology snapshot.)



Step 2: Select `VegetarianPizza`

Select `VegetarianPizza` in the class hierarchy. In a previous exercise, we have already defined this class as

```

Class: VegetarianPizza
  SubClassOf: Pizza
  SubClassOf: hasBase some PizzaBase
  EquivalentTo: hasTopping only VegetableTopping

```

We will update it now to match **Exercise 22**.

The screenshot shows the Protege interface for the ontology 'pizza-ebook_ex22'. The 'Classes' tab is active, displaying the 'Class hierarchy: VegetarianPizza'. The hierarchy shows 'VegetarianPizza' as a subclass of 'Pizza'. The 'Annotations' tab is also visible, showing the 'rdfs:label' as 'Vegetarian Pizza' and the 'rdfs:comment' as 'A pizza containing only non-meat toppings, suitable for vegetarian diets.' The 'Description' tab shows 'VegetarianPizza' with 'Equivalent To' and 'SubClass Of' sections. The 'SubClass Of' section shows 'VegetarianPizza' as a subclass of 'Pizza'.

Step 3: Add the universal restriction

1. In the Description view, locate the **SubClassOf** section
2. Click the Add icon (+)
3. In the Class Expression Editor, enter `hasTopping only (CheeseTopping or VegetableTopping)`
4. Click OK

After this step, `VegetarianPizza` has:

```
Class: VegetarianPizza
SubClassOf: Pizza
SubClassOf: hasTopping only (CheeseTopping or VegetableTopping)
```

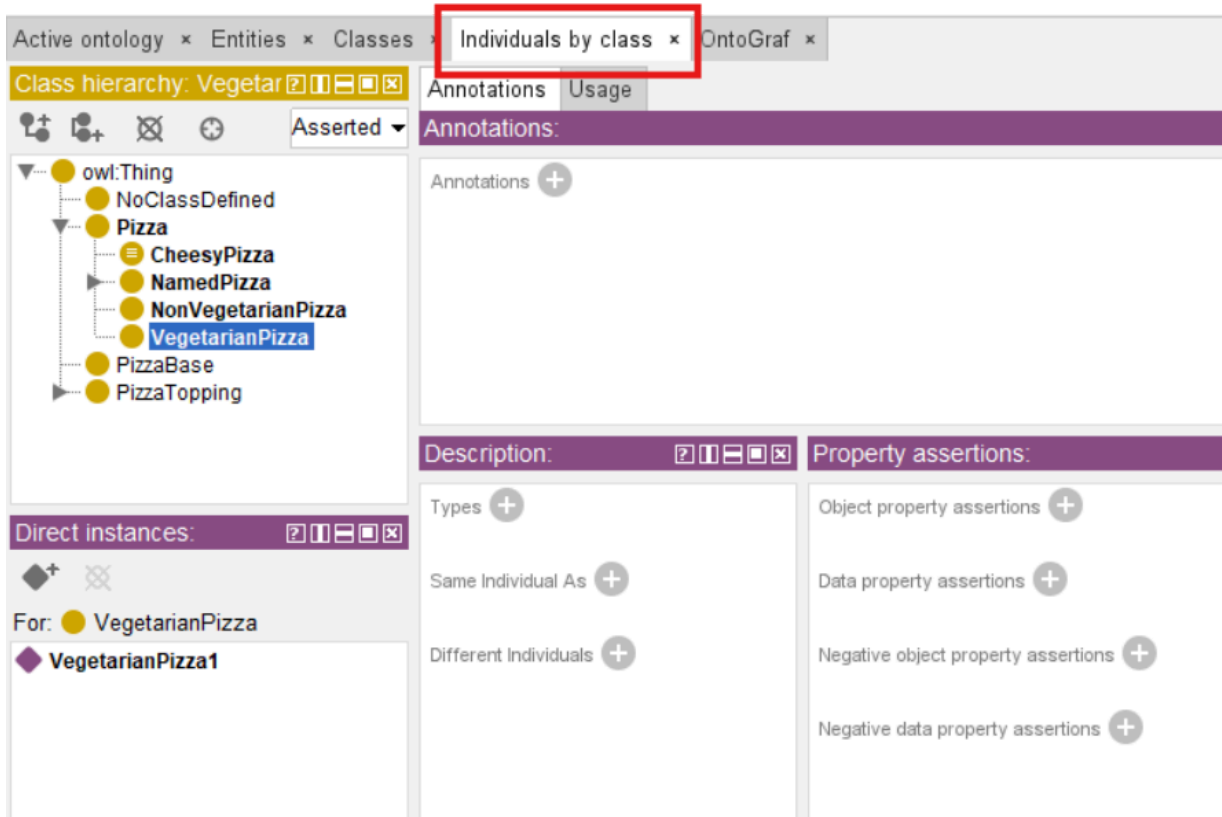
The ontology after this step is as below:

The screenshot shows the Protege interface after adding the universal restriction. The 'Classes' tab is active, displaying the 'Class hierarchy: VegetarianPizza'. The hierarchy shows 'VegetarianPizza' as a subclass of 'Pizza'. The 'Annotations' tab is also visible, showing the 'rdfs:label' as 'Vegetarian Pizza' and the 'rdfs:comment' as 'A pizza containing only non-meat toppings, suitable for vegetarian diets.' The 'Description' tab shows 'VegetarianPizza' with 'Equivalent To' and 'SubClass Of' sections. The 'SubClass Of' section shows 'VegetarianPizza' as a subclass of 'Pizza' and 'hasTopping only (CheeseTopping or VegetableTopping)'. The 'hasTopping only (CheeseTopping or VegetableTopping)' restriction is highlighted with a red box.

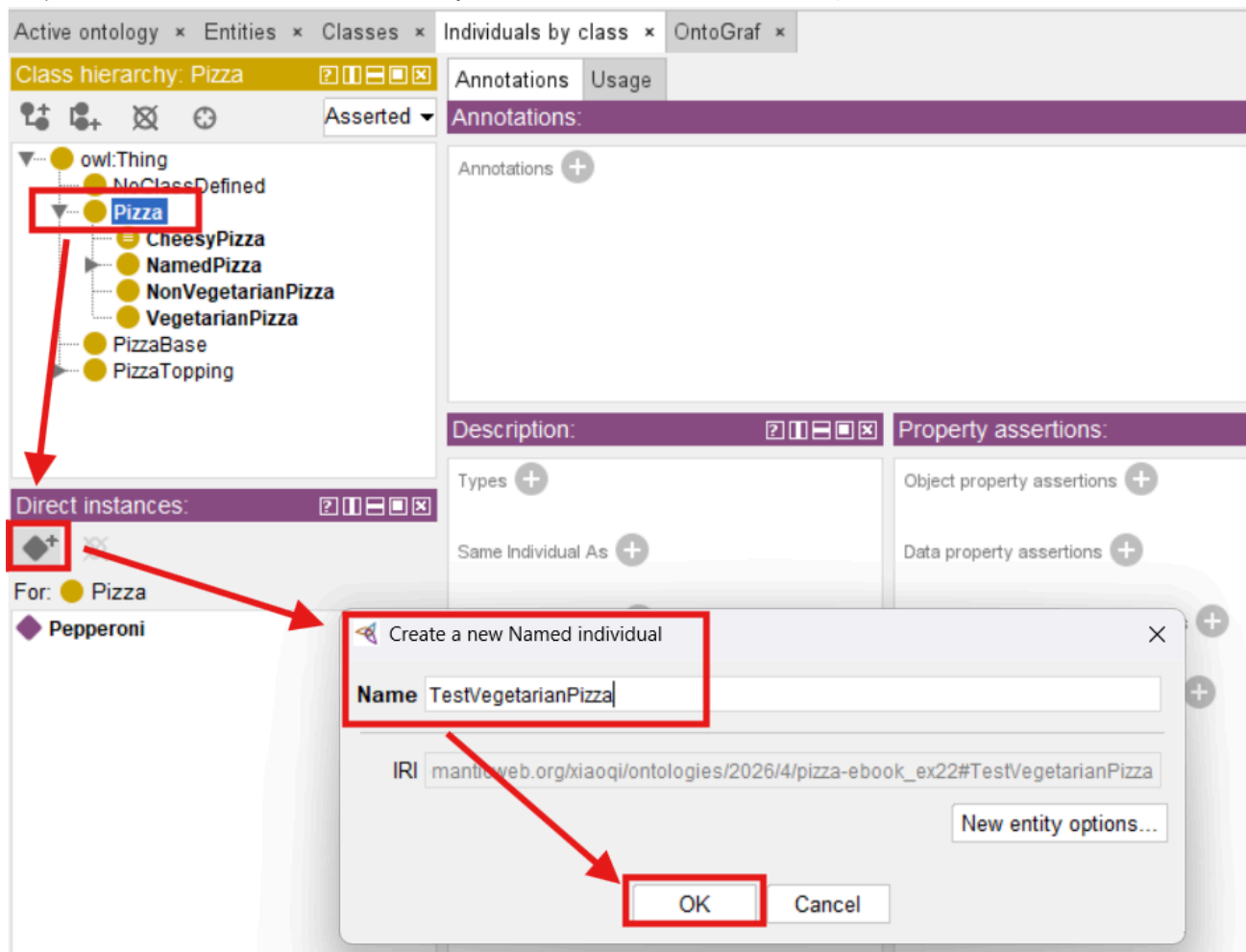
Step 4: Add a test individual

To observe the effect of the universal restriction, let's add a test individual.

1. Navigate to the `Individuals by class` tab



2. Keep class `Pizza` selected in the class hierarchy, create a new individual named `TestVegetarianPizza`



3. After click `ok`, notice the type of individual is `Pizza` now

The screenshot shows the OntoGraf interface with the following components:

- Active ontology** tab: Shows the class hierarchy for **Pizza**. The hierarchy includes:
 - owl:Thing
 - NoClassDefined
 - Pizza (selected)
 - CheesyPizza
 - NamedPizza
 - NonVegetarianPizza
 - VegetarianPizza
 - PizzaBase
 - PizzaTopping
- Annotations** tab: Shows the annotations for **TestVegetarianPizza**. The **Types** section is highlighted with a red box, showing **Pizza** as the type.
- Property** tab: Shows the property **hasTopping** with the value **some MozzarellaTopping**.

4. Add the property assertion `hasTopping some MozzarellaTopping`

The screenshot shows the OntoGraf interface with the following components:

- Direct instances** tab: Shows the instance **TestVegetarianPizza** for the class **Pizza**.
- Property assertions** tab: Shows the property assertion **hasTopping some MozzarellaTopping** for the instance **TestVegetarianPizza**. The **Class expression editor** is highlighted with a red box, showing the expression `hasTopping some MozzarellaTopping`.

5. Add the property assertion `hasTopping some TomatoTopping`

The screenshot shows the OntoGraf interface with the following components:

- Direct instances** tab: Shows the instance **TestVegetarianPizza** for the class **Pizza**.
- Property assertions** tab: Shows the property assertion **hasTopping some TomatoTopping** for the instance **TestVegetarianPizza**. The **Class expression editor** is highlighted with a red box, showing the expression `hasTopping some TomatoTopping`.

6. Now ensure your ontology is looked like this

Active ontology

Entities

Classes

Individuals by class

OntoGraf

Class hierarchy: VegetarianPizza

Annotations

Usage

Annotations: TestVegetarianPizza

Description: TestVegetarianPizza

Types

owl:Thing

NoClassDefined

Pizza

CheesyPizza

NamedPizza

NonVegetarianPizza

VegetarianPizza

PizzaBase

PizzaTopping

Direct instances: TestVegetarian1

For: VegetarianPizza

VegetarianPizza1

Annotations

Description: TestVegetarianPizza

Types

hasTopping some MozzarellaTopping

hasTopping some TomatoTopping

Pizza

Same Individual As

Step 5: Synchronize the reasoner

Select Reasoner > Synchronize reasoner (Ctrl+R).

Observe the result.

Active ontology x Entities x Classes x Individuals by class x OntoGraf x

Class hierarchy: VegetarianPizza

Annotations Usage

Annotations: TestVegetarianPizza

Annotations +

Description: TestVegetarianPizza

Property assertions

Types +

hasTopping	some	MozzarellaTopping	?	@	x	o
hasTopping	some	TomatoTopping	?	@	x	o
		Pizza	?	@	x	o
		CheesyPizza	?	@		

Same Individual As +

Direct instances: TestVegetarianPizza

For: VegetarianPizza

VegetarianPizza1

With the ontology designed so far, the reasoner classifies `TestVegetarianPizza` as a `CheesyPizza`, because we have defined `CheesyPizza` earlier using `EquivalentTo` (defined class) and `TestVegetarianPizza` has toppings `MozzarellaTopping` (a `CheeseTopping`):

```
Class: CheesyPizza
  Equivalent To: Pizza and (hasTopping some CheeseTopping)
  SubClass Of: Pizza
```

Class hierarchy: CheesyPizza

- owl:Thing
 - NoClassDefined
 - Pizza
 - CheesyPizza
 - NamedPizza
 - NonVegetarianPizza
 - VegetarianPizza
 - PizzaBase
 - PizzaTopping

Annotations: CheesyPizza

Annotations +

Description: CheesyPizza

Equivalent To +

Pizza and (hasTopping some CheeseTopping)

SubClass Of +

Pizza

But until now the `VegetarianPizza` is still under only `SubClassOf` (primitive class), so based on Open World Assumption (OWA), the reasoner cannot infer `TestVegetarianPizza` as a type of `VegetarianPizza`.

Step 6: Convert `VegetarianPizza` to a defined class

This is the crucial step. Select `VegetarianPizza` in the Classes tab, then select `Edit > Convert to defined class` (or from right-click menu).

Active ontology * Entities * Classes * Individuals by class * OntoGraf *

Class hierarchy Class hierarchy (inferred) Annotations Usage

Class hierarchy: VegetarianPizza

- owl:Thing
 - NoClassDefined
 - Pizza
 - CheesyPizza
 - NamedPizza
 - NonVegetarianPizza
 - VegetarianPizza
 - PizzaBase
 - PizzaTopping

Annotations: VegetarianPizza

Annotations +

rdfs:label

Vegetarian Pizza

rdfs:comment

A pizza containing only non-meat toppings, suitable for vegetarian diets.

Context menu:

- Add subclass... Ctrl-E
- Add sibling class... Ctrl+Shift-E
- Add subclasses...
- Duplicate class... Ctrl+Shift-C
- Change IRI (Rename)... Ctrl-U
- Copy Ctrl-C
- Copy IRI
- Copy OBO Id
- Copy display name
- Copy as Markdown
- Copy sub-hierarchy as tab indented text Ctrl+Alt-C
- Show IRI in Web browser
- Make primitive siblings disjoint Ctrl-J
- Convert to primitive class Ctrl-P
- Convert to defined class Ctrl-D
- Remove disjoint classes ax Converts the superclasses of the selected class to an equivalent class.

Observe what happens in the Description view. The `SubClassOf` axiom for the universal restriction is moved to `EquivalentTo`:

Class: VegetarianPizza
 Equivalent To: Pizza and (hasTopping only (CheeseTopping or VegetableTopping))
 SubClass Of: Pizza

The screenshot shows the Protege OWL editor with the 'Classes' tab selected. The left pane displays the 'Class hierarchy' for 'VegetarianPizza', showing it as a subclass of 'Pizza'. The right pane shows the 'Annotations' for 'VegetarianPizza', including an 'rdfs:label' and an 'rdfs:comment'. The 'Description' pane for 'VegetarianPizza' is highlighted with a red box, showing the equivalence: 'Equivalent To: Pizza and (hasTopping only (CheeseTopping or VegetableTopping))'. The 'SubClass Of' pane shows 'Pizza' as the superclass.

Note

The `VegetarianPizza` is now shown with three horizontal stripes in the class icon -- the visual indicator that it is a defined class.

Tip

Following the Tutorial

The conversion you just performed -- Edit > Convert to defined class on `VegetarianPizza` -- is exactly **Step 4 of Exercise 22** in the original `Pizza` Tutorial. In the tutorial, this step occurs *after* adding the universal restriction and *before* running the reasoner.

In this book, we have re-ordered the steps slightly: we added `TestVegetarianPizza` *before* converting to a defined class, so that you can observe the OWA problem as it happens -- first without the conversion (Step 5), and then with it (Step 7-8). Both orderings produce the same final ontology state. The tutorial's minimal path is: add restriction → convert to defined class → synchronize. Our path adds the test individuals in between, to make the OWA lesson visible.

If you are following along with Michael DeBellis' tutorial in parallel, you can safely perform the conversion now as Step 4 of the tutorial, and then return to observe the OWA behavior when you reach Step 8 in our book.

Step 7: Synchronize the reasoner again

Select Reasoner > Synchronize reasoner (Ctrl+R)

Step 8 -- Observe the result -- and be surprised!

Look at `TestVegetarianPizza` in the **Individual by class** tab.

The screenshot shows the Protege ontology editor interface. On the left, the 'Class hierarchy: Pizza' pane displays a tree structure starting from 'owl:Thing', with 'Pizza' as a subclass. Under 'Pizza', there are subclasses like 'CheesyPizza', 'NamedPizza', 'NonVegetarianPizza', and 'VegetarianPizza'. The 'Direct instances: TestVegetarianPizza' pane shows instances for 'Pepperoni' and 'TestVegetarianPizza'. The 'Description: TestVegetarianPizza' pane shows the definition: 'hasTopping some MozzarellaTopping' and 'hasTopping some TomatoTopping'.

You will see that the reasoner has kept classifying `TestVegetarianPizza` as:

- ☒ `CheesyPizza` (because it has at least one cheese topping -- mozzarella)

But it has **NOT** classified `TestVegetarianPizza` as:

- ☒ `VegetarianPizza`

This is surprising. `TestVegetarianPizza` has only mozzarella and tomato toppings. Both are vegetarian. **Why is it not a `VegetarianPizza`?**

Step 9 -- Pause and understand

This result is not a bug. It is still the **Open World Assumption (OWA)** in action, although we have converted `VegetarianPizza` from primitive class to defined class, and it is one of the most important lessons in ontology engineering.

When we define `VegetarianPizza` as:

```
Pizza and (hasTopping only (CheeseTopping or VegetableTopping))
```

we are saying: **"All toppings must be `CheeseTopping` OR `VegetableTopping`."**

But `TestVegetarianPizza` only has two *asserted* toppings. Under the Open World Assumption, the reasoner cannot conclude that these are the *only* toppings -- there might be other toppings that exist in the world (somewhere, anywhere) not been added to the ontology yet.

The reasoner can only classify an individual into a defined class when it has enough information to prove that the individual satisfies the definition. And it does not have enough information here.

Why `CheesyPizza` works but `VegetarianPizza` doesn't:

Class	Definition	Reasoner behavior
<code>CheesyPizza</code>	<code>Pizza and (hasTopping some CheeseTopping)</code>	Existential restriction. The reasoner only needs to find one cheese topping -- which exists (<code>MozzarellaTopping</code>). ☒ Classified.
<code>VegetarianPizza</code>	<code>Pizza and (hasTopping only (CheeseTopping or VegetableTopping))</code>	Universal restriction. The reasoner must verify that all toppings satisfy the constraint -- but OWA prevents it from knowing that no

Class	Definition	Reasoner behavior
		other toppings exist. ✗ Not classified.

Something unexpected has happened. `TestVegetarianPizza` has only mozzarella topping (a `CheeseTopping`) and tomato topping (a `VegetableTopping`). We defined `VegetarianPizza` as a defined class using a universal restriction. And yet the reasoner has classified `TestVegetarianPizza` as a `CheesyPizza` but **not** as a `VegetarianPizza`.

This is not a bug in Protégé, and it is even not an error in our ontology. It is a fundamental property of OWL that every ontology engineer must understand. The reason for this behavior -- and the construct that resolves it -- is the subject of the next exercise.

24.5.3 The Result: What the Reasoner Does Not Do

When the reasoner is synchronized, something important -- or rather, something important does **not** happen.

What to observe: (let's recap)

- `CheesyPizza` is correctly inferred for `TestVegetarianPizza` (highlighted in yellow, indicating a reasoner inference).
- `TestVegetarianPizza` is **not** classified as a `VegetarianPizza`.

The reasoner has all the information it needs to conclude that `TestVegetarianPizza` is a `VegetarianPizza` -- it has mozzarella (a `CheeseTopping`) and tomato (a `VegetableTopping`), both of which are permitted by the universal restriction on `VegetarianPizza`. **And yet it does not make that inference.**

This is unexpected. We defined `VegetarianPizza` as a defined class with the universal restriction `hasTopping only (CheeseTopping or VegetableTopping)`. We created `TestVegetarianPizza` with exactly the kinds of toppings that restriction permits. Intuitively, from human being's perspective, the reasoner should classify it as a `VegetarianPizza`.

But it does not.

Key Insight:

"The universal restriction tells the reasoner what is allowed. It does not tell the reasoner what is *exhaustively* present. This distinction is the difference between a description and a specification.

Why this happens -- and what construct resolves it -- is the subject of **Exercise 23**.

24.5.4 Why the Universal Restriction Is Not Enough

To understand why the reasoner does not classify `TestVegetarianPizza` as a `VegetarianPizza`, we need to examine how universal restrictions behave under the **Open World Assumption (OWA)**.

The universal restriction:

```
VegetarianPizza
  EquivalentTo
    Pizza and (hasTopping only (CheeseTopping or VegetableTopping))
```

means: "If something is a `VegetarianPizza`, then every one of its toppings must be either a `CheeseTopping` or a `VegetableTopping`."

For the reasoner to conclude the `TestVegetarianPizza` **is** a `VegetarianPizza`, it must verify that the universal restriction holds -- that *all* of `TestVegetarianPizza`'s toppings are cheese or vegetable. But under the Open World Assumption, the reasoner cannot assume that the two toppings it knows about are the *only* toppings. There might be other toppings that exist but have not been asserted.

So the reasoner cannot prove that `TestVegetarianPizza` satisfies the universal restriction, and therefore cannot classify it as a `VegetarianPizza`.

This is not a flaw in the ontology. It is a faithful consequence of how OWL is designed. The Open World Assumption is what makes OWL suitable for open, federated domains -- but it is also what makes universal restrictions insufficient for classification on their own.

Key Insight:

"The universal restriction tells the reasoner: 'If you find a `VegetarianPizza` , check that all its toppings are either cheese or vegetable.' It does not tell the reasoner: 'If you find a pizza whose known toppings are all cheese or vegetable, it is a `VegetarianPizza` .' The second statement requires more information than the universal restriction alone can supply."

24.5.5 Engineering Perspective: Exercise 22 as a Case-Study

From the perspective of Protégé, **Exercise 22** involves adding a single restriction and creating a test individual. (Note: the addition of the test individual is a book-level extension to the tutorial for triggering more discussion.) It appears to be a small modeling activity.

From the perspective of ontology engineering, however, something significant has occurred. The exercise has introduced the first execution primitive -- the universal restriction -- *and* surfaced the first significant engineering obstacle:

the OWA between what a universal restriction describes and what execution requires.

The engineering contribution of **Exercise 22** can be summarized as follows:

Engineering Perspective	Contribution from Exercise 22
SKDL Stage	Stage 7 -- Executable Knowledge
Primary Objective	Introduce the universal restriction as an execution primitive
Modeling Activity	Add <code>hasTopping only (VegetableTopping or CheeseTopping)</code> to <code>VegetarianPizza</code> ; convert to defined class; add <code>TestVegetarianPizza</code> individual
Engineering Outcome	Demonstrate the universal restriction's expressive power -- and its limitation under OWA
Preparation for Later Sections	Set up the OWL problem that Exercise 23 will resolve through closure axioms

Key Insight:

"**Exercise 22** is a lesson in what the universal restriction can and cannot do. It can **exclude**. It cannot, on its own, **prove** exhaustiveness. The task belongs to the closure axiom."

24.6 Exercise 23 -- Closure Axioms and the Open World Assumption

In **Exercise 22**, we saw a surprising result: even though `VegetarianPizza` was correctly defined with a universal restriction, the reasoner did not classify `TestVegetarianPizza` as a `VegetarianPizza` . The reason was the Open World Assumption (OWA) -- the reasoner could not conclude that `TestVegetarianPizza` has *only* mozzarella and tomato toppings, because it could not rule out the existence of other toppings that had not been asserted.

This is the same phenomenon that Michael DeBellis demonstrates in Section 4.13 of the `pizza` Tutorial, where he shows that `MargheritaPizza` and `SohoPizza` -- despite having only cheese and vegetable toppings -- are not automatically classified as subclasses of `VegetarianPizza` by the reasoner.

The construct that resolves this problem is called the **closure axiom**.

This exercise follows the tutorial's **Exercise 23**, and then extends it slightly to tie back to the `TestVegetarianPizza` individual from the previous exercise.

24.6.1 The Engineering Objective

Exercise 23 introduces the **closure axiom** -- a universal restriction that explicitly specifies the complete set of values a property may take for a class or an individual.

A closure axiom for `MargheritaPizza` would be:

```
Class: MargheritaPizza
SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)
```

This says: "Every topping on a `MargheritaPizza` must be either `MozzarellaTopping` or `TomatoTopping` , and no other toppings exist."

By adding this axiom, we close the world for `MargheritaPizza` . The reasoner now knows that the two toppings we asserted for `MargheritaPizza` are the *only* toppings it has. This is the information it needs to classify `MargheritaPizza` as a subclass of `VegetarianPizza` .

Key Insight:

- "A closure axiom is the OWL equivalent of saying 'this is the complete picture.'
- Without closure, the reasoner remains cautious.
 - With closure, the reasoner can be decisive."

24.6.2 Implementation: Adding Closure Axioms to MargheritaPizza and SohoPizza

We begin with the tutorial's **Exercise 23**, including the discussion in tutorial's Section 4.13 (before the exercise), which applies closure axioms at the class level.

Step 1: Open the pizza ontology

Open `pizza-ebook_ex22.rdf` in Protégé -- the file from the previous exercise, which already contains `TestVegetarianPizza` and the inferred-but-incomplete classification we observed.

FileEditViewReasonerToolsRefactorWindowHelp

pizza-ebook_ex22 (http://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook_ex22) : [D:\github\protege_pizza\ebook\rdf\pizza-ebook_ex23.rdf]

Active ontology x Entities x Classes x Individuals by class x OntoGraf x

Ontology header:

Ontology IRIhttp://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook_ex22

Ontology Version IRIe.g. http://www.semanticweb.org/xiaoqi/ontologies/2026/4/pizza-ebook_ex22/r1.0.0

Annotations +

dcterms:license [type: xsd:anyURI]
https://creativecommons.org/licenses/by-sa/4.0/

Ontology metrics:

Metrics

Axiom	136
Logical axiom count	76
Declaration axioms count	51
Class count	30
Object property count	4
Data property count	0
Individual count	16
Annotation Property count	3

Class axioms

SubClassOf	39
EquivalentClasses	2
DisjointClasses	4
GCI count	0
Hidden GCI Count	0

Ontology imports

Ontology Prefixes

General class axioms

Imported ontologies:

Note

The finished state of **Exercise 23** will be saved as `pizza-ebook_ex23.rdf` .

Step 2: Select MargheritaPizza

In the Classes tab, select `MargheritaPizza` in the class hierarchy.

Active ontology x Entities x **Classes** x Individuals by class x OntoGraf x

Class hierarchy Class hierarchy (inferred) Annotations Usage

Class hierarchy: MargheritaPizza

Annotations: MargheritaPizza

Annotations +

rdfs:comment
A pizza that only has Mozzarella and Tomato toppings

Description: MargheritaPizza

Equivalent To +

SubClass Of +

- hasTopping some MozzarellaTopping
- hasTopping some TomatoTopping
- NamedPizza

So far, the `MargheritaPizza` is defined as:

```
Class: MargheritaPizza
SubClassOf: NamedPizza
SubClassOf: hasTopping some MozzarellaTopping
SubClassOf: hasTopping some TomatoTopping
```

Step 3: Add the closure axiom to `MargheritaPizza`

1. In the Description view, locate the **SubClass Of** section

Active ontology x Entities x **Classes** x Individuals by class x OntoGraf x

Class hierarchy Class hierarchy (inferred) Annotations Usage

Class hierarchy: MargheritaPizza

Annotations: MargheritaPizza

Annotations +

rdfs:comment
A pizza that only has Mozzarella and Tomato toppings

Description: MargheritaPizza

Equivalent To +

SubClass Of +

- hasTopping some MozzarellaTopping
- hasTopping some TomatoTopping
- NamedPizza

2. Click the Add icon (+)

The screenshot shows the Protégé interface with the 'Class hierarchy' tab selected. The hierarchy for 'MargheritaPizza' is displayed on the left, showing it as a subclass of 'NamedPizza'. The 'Annotations' tab is also visible, showing an 'rdfs:comment' for 'MargheritaPizza' with the text 'A pizza that only has Mozzarella and Tomato toppings'. The 'SubClass Of' section is highlighted with a red box, showing 'MargheritaPizza' as a subclass of 'NamedPizza'.

3. In the Class Expression Editor, enter: `hasTopping only (MozzarellaTopping or TomatoTopping)`

The screenshot shows the 'Class expression editor' tab for 'MargheritaPizza'. The expression `hasTopping only (MozzarellaTopping or TomatoTopping)` is entered in the text field. The 'Class hierarchy' tab is also visible, showing the hierarchy for 'MargheritaPizza'.

4. Click ok .

After this step, `MargheritaPizza` has the following description:

```

Class: MargheritaPizza
SubClassOf: NamedPizza
SubClassOf: hasTopping some MozzarellaTopping
SubClassOf: hasTopping some TomatoTopping
SubClassOf: hasTopping only (MozzarellaTopping or TomatoTopping)

```

Class hierarchy: MargheritaPizza

Class hierarchy (inferred)

- owl:Thing
 - NoClassDefined
 - Pizza
 - CheesyPizza
 - NamedPizza
 - AmericanaPizza
 - MargheritaPizza**
 - NonVegetarianPizza
 - VegetarianPizza
 - PizzaBase
 - PizzaTopping

Annotations: MargheritaPizza

Annotations

rdfs:comment

A pizza that only has Mozzarella and Tomato toppings

Description: MargheritaPizza

Equivalent To

SubClass Of

- hasTopping **only** (MozzarellaTopping or TomatoTopping)
- hasTopping **some** MozzarellaTopping
- hasTopping **some** TomatoTopping
- NamedPizza

Step 4: Add the closure axiom to SohoPizza

Repeat Step 2-3, but this time select `SohoPizza` and enter the axiom:

hasTopping only (MozzarellaTopping or TomatoTopping or ParmesanTopping or OliveTopping)

Class hierarchy: SohoPizza

Class hierarchy (inferred)

- owl:Thing
 - NoClassDefined
 - Pizza
 - CheesyPizza
 - NamedPizza
 - AmericanaPizza
 - MargheritaPizza
 - SohoPizza**
 - NonVegetarianPizza
 - VegetarianPizza
 - PizzaBase
 - PizzaTopping

Annotations: SohoPizza

Annotations

rdfs:comment

A pizza that has Mozzarella, Tomato, Olive, and Parmesan toppings.
Created by cloning from MargheritaPizza.
Additional restrictions: hasTopping some OliveTopping, hasTopping some ParmesanTopping.
Source: MargheritaPizza.

Description: SohoPizza

Equivalent To

SubClass Of

- hasTopping **some** MozzarellaTopping
- hasTopping **some** OliveTopping
- hasTopping **some** ParmesanTopping
- hasTopping **some** TomatoTopping
- MargheritaPizza

SohoPizza

Class expression editor

hasTopping only (MozzarellaTopping or TomatoTopping or ParmesanTopping or OliveTopping)

After this step, `SohoPizza` has the closure axiom that enumerates its full set of allowed toppings.

Class hierarchy: SohoPizza

- owl:Thing
 - NoClassDefined
 - Pizza
 - CheesyPizza
 - NamedPizza
 - AmericanaPizza
 - MargheritaPizza
 - SohoPizza**
 - NonVegetarianPizza
 - VegetarianPizza
 - PizzaBase
 - PizzaTopping

Annotations: SohoPizza

rdfs:comment
A pizza that has Mozzarella, Tomato, Olive, and Parmesan toppings.
Created by cloning from MargheritaPizza.
Additional restrictions: hasTopping some OliveTopping, hasTopping some ParmesanTopping.
Source: MargheritaPizza.

Description: SohoPizza

Equivalent To +

SubClass Of +

- hasTopping only**
(MozzarellaTopping or OliveTopping or ParmesanTopping or TomatoTopping)
- hasTopping some MozzarellaTopping
- hasTopping some OliveTopping
- hasTopping some ParmesanTopping
- hasTopping some TomatoTopping
- MargheritaPizza

Step 5: Synchronize the reasoner

Select Reasoner > Synchronize reasoner (Ctrl+R)

Step 6: Observe the result

Look at the **Class hierarchy (inferred)** tab.

The reasoner now classifies both `MargheritaPizza` and `SohoPizza` (which is still a subclass of `MargheritaPizza`) as subclasses of `VegetarianPizza`.

This is the result we were looking for:

Active ontology x **Entities** x **Classes** x **Individuals by class** x **OntoGraf** x

Class hierarchy: VegetarianPizza

- owl:Thing
 - owl:Nothing
 - NoClassDefined
 - Pizza
 - CheesyPizza
 - NamedPizza
 - AmericanaPizza
 - MargheritaPizza
 - SohoPizza
 - NonVegetarianPizza
 - VegetarianPizza**
 - PizzaBase
 - PizzaTopping

Annotations: VegetarianPizza

rdfs:label
Vegetarian Pizza

rdfs:comment
A pizza containing only non-meat toppings, suitable for vegetarian diets.

Description: VegetarianPizza

Equivalent To +

- Pizza
and (hasTopping only
(CheeseTopping or VegetableTopping))

SubClass Of +

- Pizza

Why does it work now? Because the closure axiom supplies the missing information (knowledge) -- it tells the reasoner that *only* the enumerated toppings exist for these classes. Without that information, the reasoner could not verify that the universal restriction on `VegetarianPizza` held for `MargheritaPizza` and `SohoPizza`.

Key Insight:

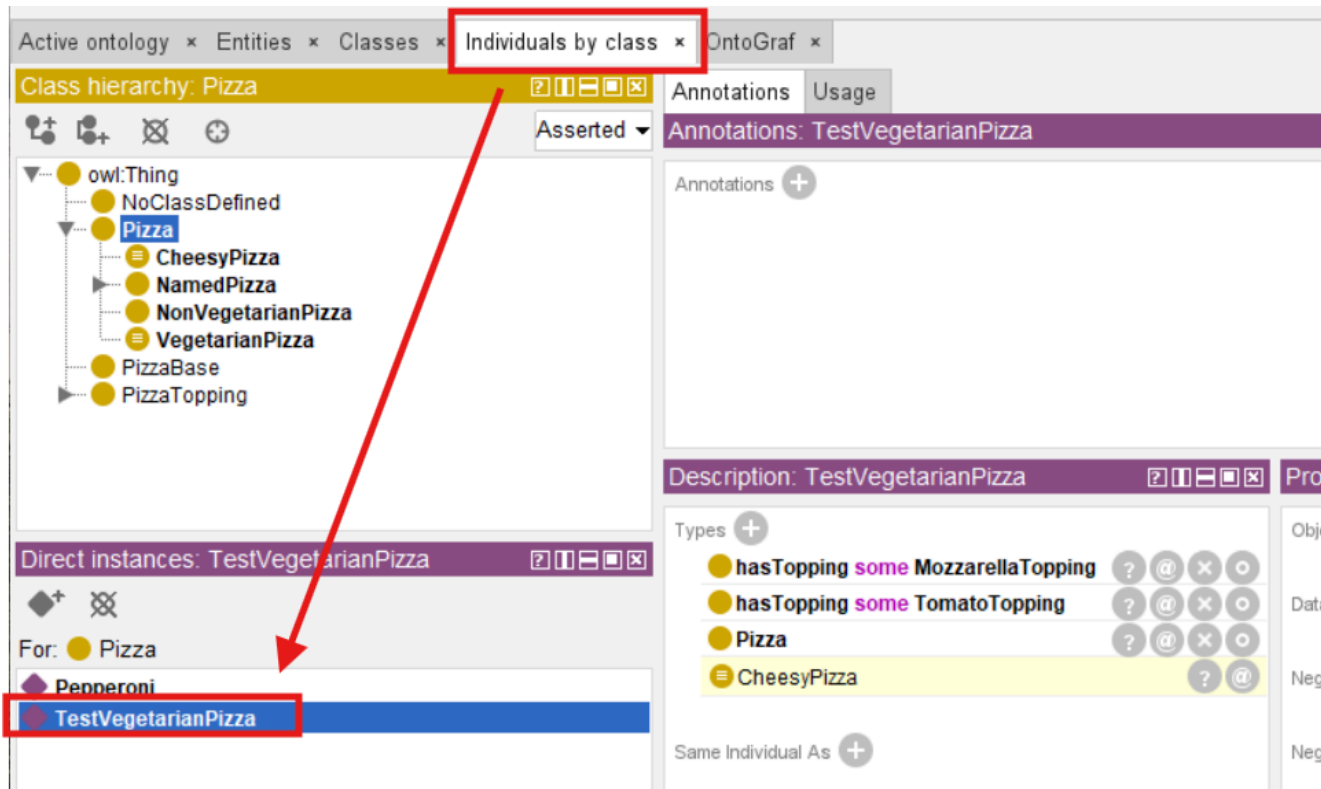
"The reasoner did not change. The definition of `VegetarianPizza` did not change. Only the **world** changed -- from **open** to **closed**. Once the world is closed, the reasoner can make the inference."

24.6.3 Extending the Demonstration: Closure at the Individual Level

We have now seen closure axioms applied at the **class** level, following the tutorial. But our `TestVegetarianPizza` individual from **Exercise 22** is still unclassified. Let's resolve that too, using the same principle.

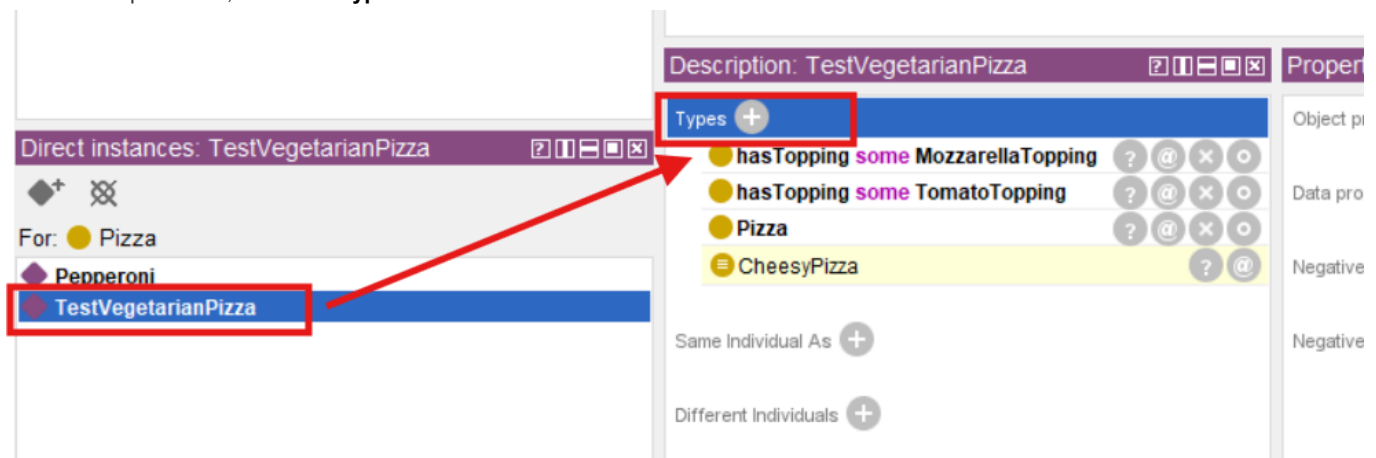
Step 1: Select `TestVegetarianPizza` individual

Navigate to the **Individual by class** tab and select `TestVegetarianPizza`.

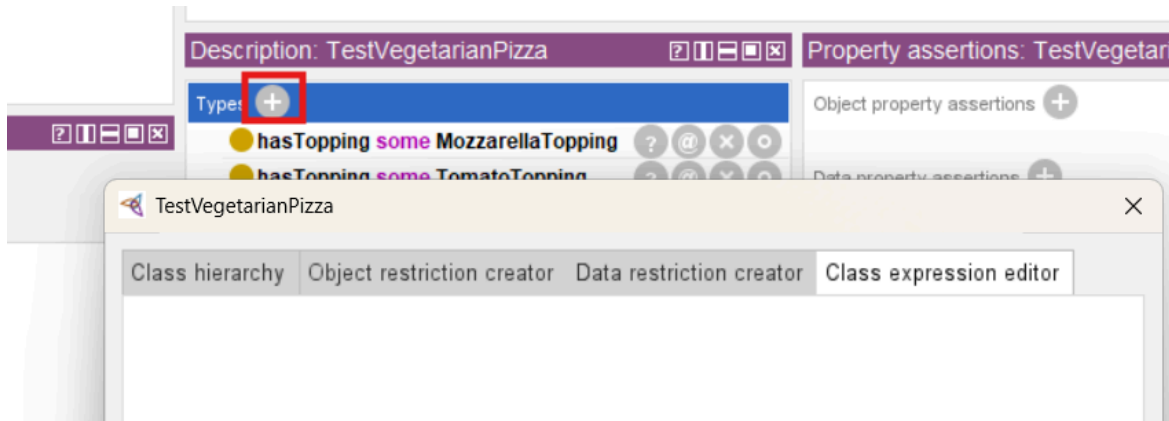


Step 2: Add the closure axiom

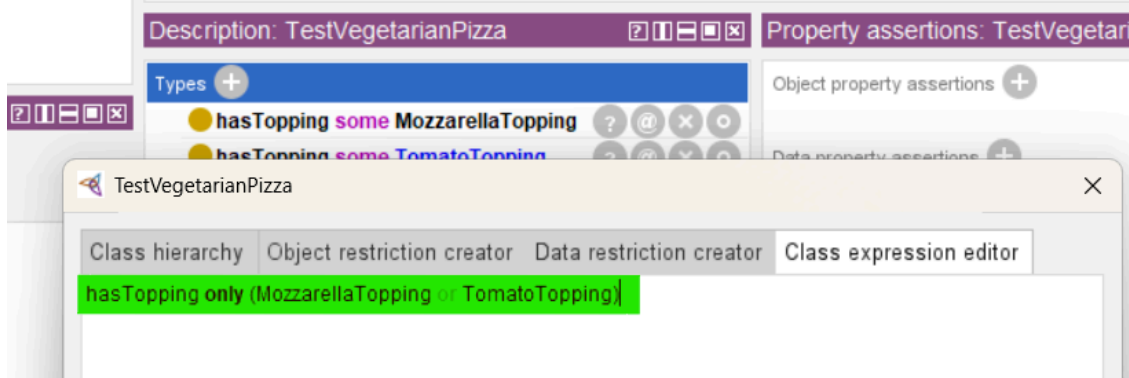
1. In the Description view, locate the **Types** section



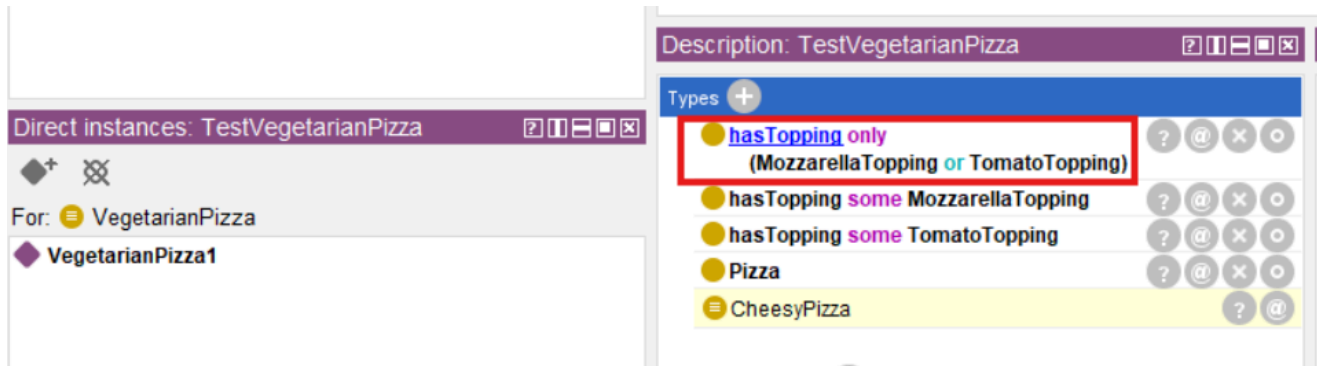
2. Click the Add icon (+)



3. In the Class Expression Editor, enter: `hasTopping only (MozzarellaTopping or TomatoTopping)`



4. Click ok



Step 3: Synchronize the reasoner

Select Reasoner > Synchronize reasoner (Ctrl+R)

Step 4: Observe the result

Active ontology × Entities × Classes × Individuals by class × OntoGraf ×

Class hierarchy: Pizza

owl:Thing

NoClassDefined

Pizza

CheesyPizza

NamedPizza

NonVegetarianPizza

VegetarianPizza

PizzaBase

PizzaTopping

Annotations

Usage

Annotations: TestVegetarianPizza

Annotations +

Description: TestVegetarianPizza

Property a

Types +

hasTopping only (MozzarellaTopping or TomatoTopping)

hasTopping some MozzarellaTopping

hasTopping some TomatoTopping

Pizza

CheesyPizza

VegetarianPizza

Object propert

Data propert

Negative obj

Negative dat

Direct instances: TestVegetarianPizza

For: Pizza

Pepperoni

TestVegetarianPizza

TestVegetarianPizza is now classified as a VegetarianPizza , along with CheesyPizza . The closure axiom has resolved the same OWA problem at the individual level that it resolved at the class level.

This confirms that closure axioms are effective at both levels. In production ontologies, they are usually applied at the class level (as we did for MargheritaPizza and SohoPizza) so that every instance of the class inherits the closure. But the same principle applies to individual-level closure when a one-off case requires it.

Key Insight:

"Closure axioms are a general mechanism. They close the world for whatever they are attached to -- a class or an individual. The reasoning principle is the same."

24.6.4 From Description to Specification

Both the class-level and individual-level closure axioms have converted descriptions into specifications.

Before the closure axiom, the reasoner could say: "If you find a MargheritaPizza , it has mozzarella and tomato." It could not say: "If you find a pizza with mozzarella and tomato, it is a MargheritaPizza ."

After the closure axiom, the reasoner can say both. The ontology is now deterministic for these classes and individuals.

Key Insight:

"The closure axiom is the bridge from description to specification. Before it, the ontology describes. After it, the ontology specifies. Specification is what enables execution."

24.6.5 Why Closure Axioms Matter for Execution

Closure axioms are essential for execution because they solve the central tension between OWA and executability:

Aspect	Without Closure Axiom	With Closure Axiom
Toppings allowed	Any topping not explicitly forbidden	Only the enumerated toppings
Reasoner behavior	Cannot exclude possibilities	Can exclude possibilities

Aspect	Without Closure Axiom	With Closure Axiom
Execution readiness	Not executable	Executable
Semantic strength	Description	Specification

The closure axiom is the OWL construct that most directly enables execution.

Key Insight:

"Every closed-world requirement in an execution system corresponds to a closure axiom in the ontology. Closure is the semantic foundation of executability."

24.6.6 Engineering Perspective: Exercise 23 as a Case Study

From the perspective of Protégé, **Exercise 23** involves adding a small number of axioms. It appears to be a minor modeling activity.

From the perspective of ontology engineering, however, something profound has occurred. The ontology has crossed the boundary from description to specification. It is no longer just telling us what is true -- it is telling us what is allowed.

The engineering contribution of **Exercise 23** can be summarized as follows:

Engineering Perspective	Contribution from Exercise 23
SKDL Stage	Stage 7 -- Executable Knowledge
Primary Objective	Introduce the closure axiom as the bridge from description to specification
Modeling Activity	Add closure axioms to <code>MargheritaPizza</code> , <code>SohoPizza</code> , and <code>TestVegetarianPizza</code>
Engineering Outcome	Convert descriptions into closed-world specifications; the reasoner now correctly classifies <code>MargheritaPizza</code> , <code>SohoPizza</code> , and <code>TestVegetarianPizza</code> as subclasses or instances of <code>VegetarianPizza</code>
Preparation for Later Chapters	Establish the closure foundation for enumerated classes, <code>hasValue</code> , and cardinality restrictions

Key Insight:

"**Exercise 23** is the moment the ontology becomes a specification. It is the most important step toward executability."

24.7 A Brief Note on OWL Design Patterns

Before we build our next execution primitive, it is worth pausing to name a concept that has quietly been present throughout this book: **the design pattern**.

The idea of a design pattern comes originally from architecture -- Christopher Alexander's *A Pattern Language* (1977) -- and was brought into software engineering by the "Gang of Four" in their 1994 book *Design Patterns*.

Alexander's insight was that good design does not before from nothing; it draws on a shared vocabulary of proven solutions to recurring problems. Each solution can be adapted to a specific context without being reinvented.

In OWL, the same idea applies. A **design pattern** is a reusable combination of OWL constructs that solves a recurring modeling problem. It sits at a higher level of abstraction than a single class or axiom -- it is a **shape**, not a thing!

By this point in the book, you have already used several OWL design patterns without necessarily calling them that:

Pattern	Introduced	Purpose
Probe Class	Chapter 22 (Exercise 19)	A deliberately inconsistent class used to test whether validation infrastructure works
Closure Axiom	Chapter 24 (Exercise 23)	A universal restriction that closes the world for a class or individual

Pattern	Introduced	Purpose
Enumerated Class	Chapter 24 (Exercise24)	A class whose extension is defined by listing all of its instances
hasValue Restriction	Chapter 24 (Exercise, upcoming)	A restriction that binds a property to a specific individual
Cardinality Restriction	Chapter 24 (Exercise 26, upcoming)	A restriction on the number of values a property may have

Each of these is a **reusable shape**, not a one-off modeling decision. You can apply the same shape to any class in any domain -- pizza, manufacturing, financing, healthcare -- and it will work the same way.

That reusability is what makes design patterns valuable for governance and execution:

- **Reuse:** A pattern written once can be instantiated many times.
- **Consistency:** Different engineers applying the same pattern produce structurally similar (or even same) models.
- **Governance:** Patterns can be named, documented, and governed like any other asset.
- **Execution:** The execution primitives in this chapter (closure, enumeration, `hasValue`, cardinality) are effectively the design patterns of Stage 7.

The full OWL design pattern landscape -- the ODP catalog ([http://www.ontologydesignpatterns.org/index.php/Ontology_Design_Patterns_.org_\(ODP\)](http://www.ontologydesignpatterns.org/index.php/Ontology_Design_Patterns_.org_(ODP))), logical vs. content patterns, pattern composition, and pattern selection strategies -- is a topic of its own. It belongs to the era of **Knowledge Graph Engineering**, when you are building systems that span multiple ontologies and reuse libraries of proven patterns.

We will return to it in **Volume 5**.

For now, the key point is this:

when you build an execution primitive in this chapter, you are not just writing an axiom -- you are instantiating a design patter.

The patters will grow with you as your ontology grows.

24.8 Exercise 24 -- Enumerated Classes and Deterministic Value Sets

Having established closure axioms in **Exercise 23**, As we saw in Section 24.7, the enumerated class is one of the design patterns of Stage 7 — here we instantiate it.

24.8.1 The Engineering Objective

Exercise 24 introduces the **enumerated class** -- a class whose extension is defined by explicitly listing all of its instances.

An enumerated class for spiciness, for example, would be:

```
Class: Spiciness
  EquivalentTo: {Hot, Medium, Mild}
```

This means: "The class `Spiciness` has exactly three instances: `Hot`, `Medium`, and `Mild`."

The engineering objective for **Exercise 24** is to demonstrate how enumeration provides a **complete, deterministic value domain** -- a prerequisite for execution.

Why does execution need enumeration?

Consider an execution rule that fires when a pizza is "spicy". For the rule to be reliable, the system must know the complete set of possible spiciness values. If the ontology only says "there exists a `Spiciness` value" but does not enumerate what that value could be, the rule cannot be exhaustively evaluated.

Enumeration closes the value domain. It says: *these ae all the possible values -- nothing more, nothing less.*

Key Insight:

"An enumerated class is a closed-world specification of a value domain. It says not just what values exist, but that **no other values exist**."

24.8.2 Implementation: Creating the Enumerated Class

Let's walk through the implementation step by step.

As you'll see, the workflow is slightly different from **Exercises 22 and 23** -- enumerated classes are created by combining three OWL constructs: a class, its three instances, and an `EquivalentTo` axiom that lists them.

Step 1: Open the `Pizza` ontology

Open `pizza-ebook_ex23.rdf` in Protégé - the file from the previous exercise, which now contains closure axioms on `MargheritaPizza` and `SohoPizza`, and the individual-level closure on `TestVegetarianPizza`.

The screenshot shows the Protégé ontology editor interface. The top menu bar includes File, Edit, View, Reasoner, Tools, Refactor, Window, and Help. The main window displays the `pizza-ebook_ex22` ontology. The left pane shows the class hierarchy, with `MargheritaPizza` selected. The right pane shows the description of `MargheritaPizza`, including its annotations and subClassOf axioms.

Class hierarchy: `MargheritaPizza`

- `owl:Thing`
 - `NoClassDefined`
 - `Pizza`
 - `CheesyPizza`
 - `NamedPizza`
 - `AmericanaPizza`
 - `MargheritaPizza` (selected)
 - `NonVegetarianPizza`
 - `VegetarianPizza`
 - `PizzaBase`
 - `PizzaTopping`

Annotations: MargheritaPizza

- `rdfs:comment`: A pizza that only has Mozzarella and Tomato toppings

Description: MargheritaPizza

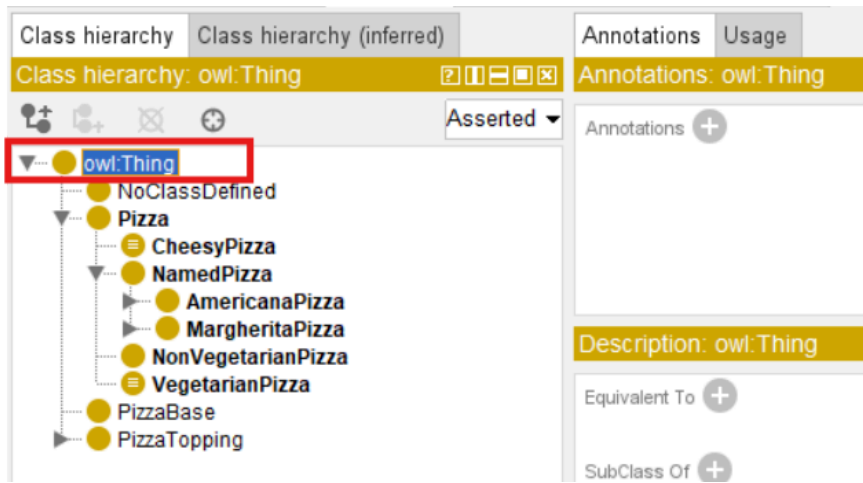
- Equivalent To**: +
- SubClass Of**: +
 - `hasTopping only (MozzarellaTopping or TomatoTopping)`
 - `hasTopping some MozzarellaTopping`
 - `hasTopping some TomatoTopping`
 - `NamedPizza`
 - `CheesyPizza`
 - `VegetarianPizza`

Note

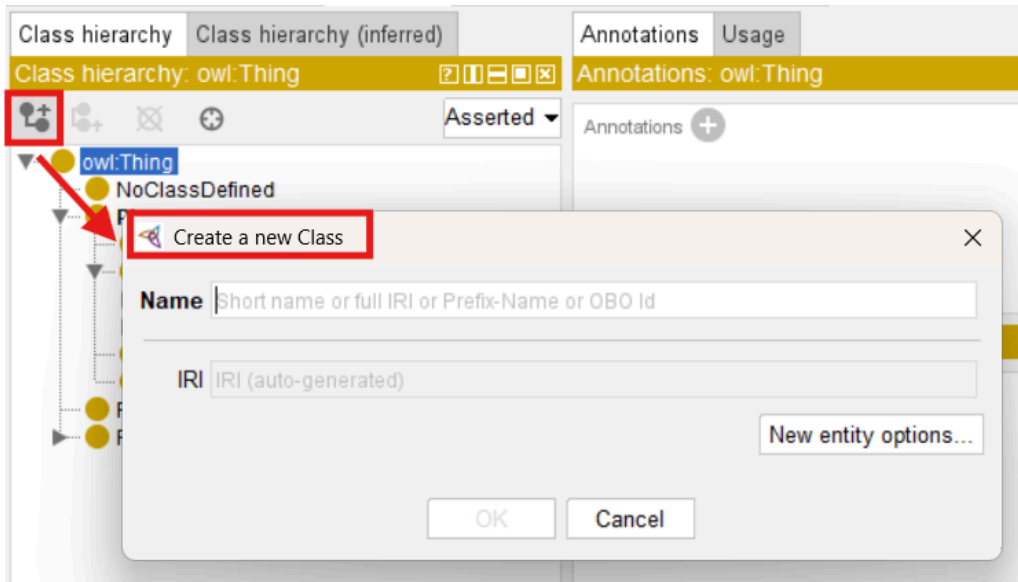
The finished state of **Exercise 24** is saved as `pizza-ebook_ex24.rdf`.

Step 2: Create the `spiciness` class

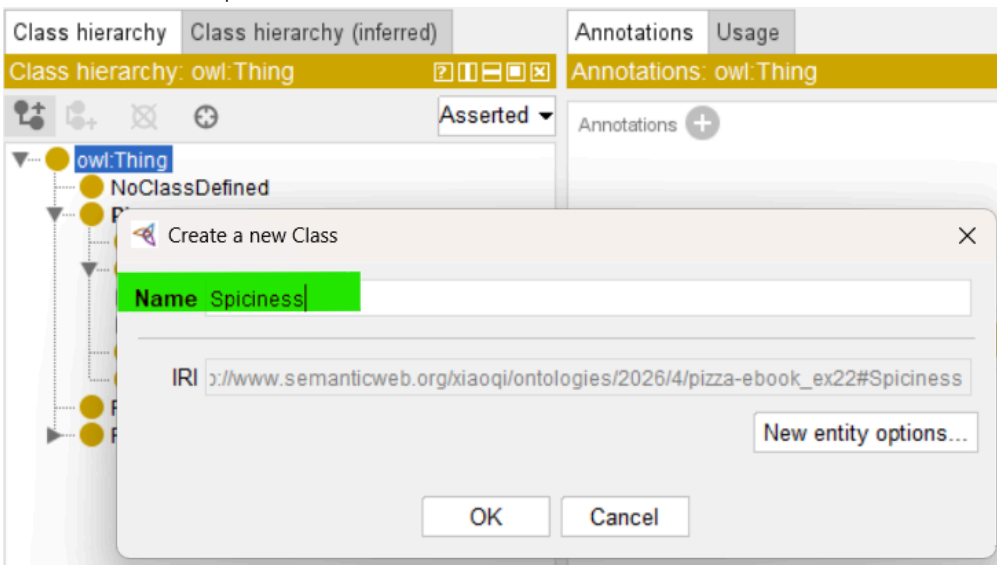
1. Navigate to the **Classes** tab. (You may already be at this tab)
2. Select `owl:Thing` in the class hierarchy



3. Click the **Add subclass** icon.



4. Name the new class **Spiciness**



5. Click **ok**

Class hierarchy: Spiciness

Annotations: Spiciness

Annotations +

Description: Spiciness

Equivalent To +

owl:Nothing

At this point, `Spiciness` is an empty class -- a class with no instances and no definition. We will now give it both.

Step 3: Create the individuals for the enumerated class

1. In the Classes tab, make sure `Spiciness` is selected. (After above Step2.5, the cursor should be by default on `Spiciness`.)
2. In the Description view, locate the `Instances` section near the bottom. (You may also switch to **Individuals by class** tab)

Active ontology x Entities x Classes x Individuals by class x OntoGraf x

Class hierarchy: Spiciness

Annotations: Spiciness

Annotations +

Description: Spiciness

Equivalent To +

owl:Nothing

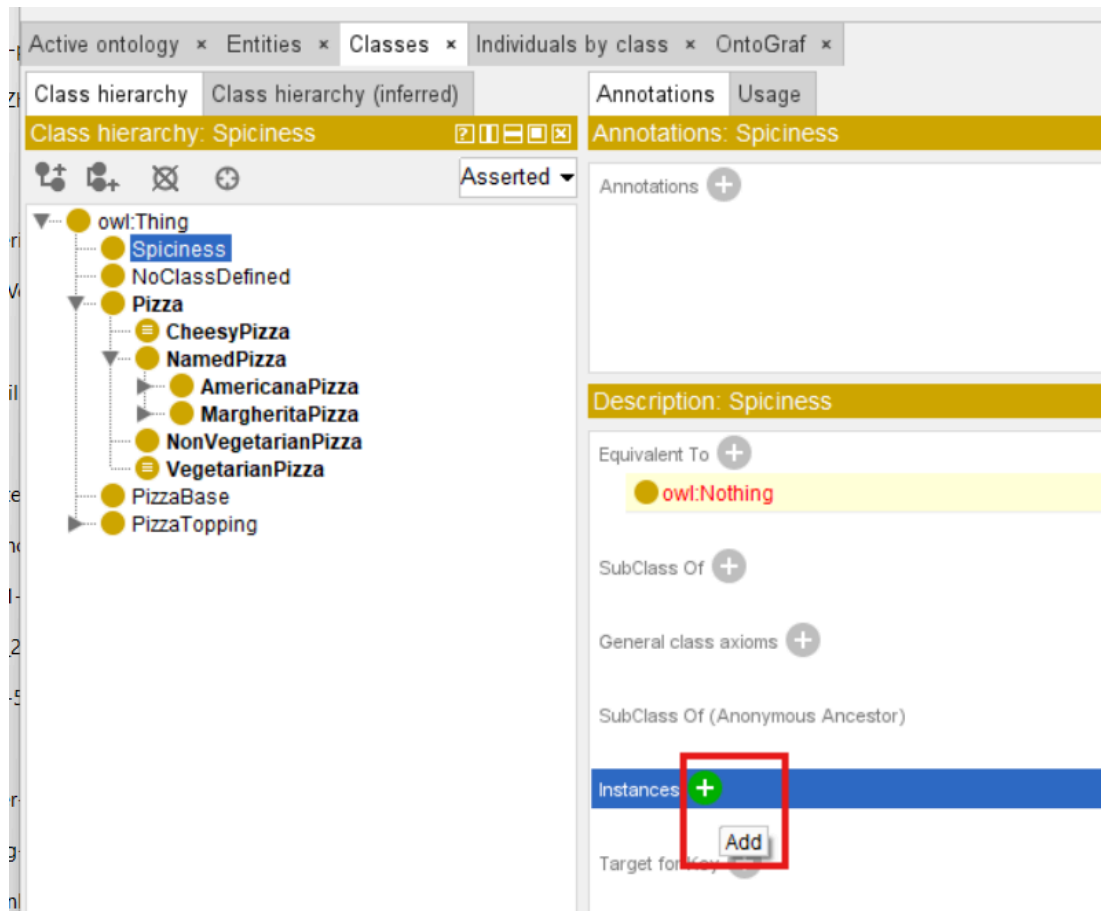
SubClass Of +

General class axioms +

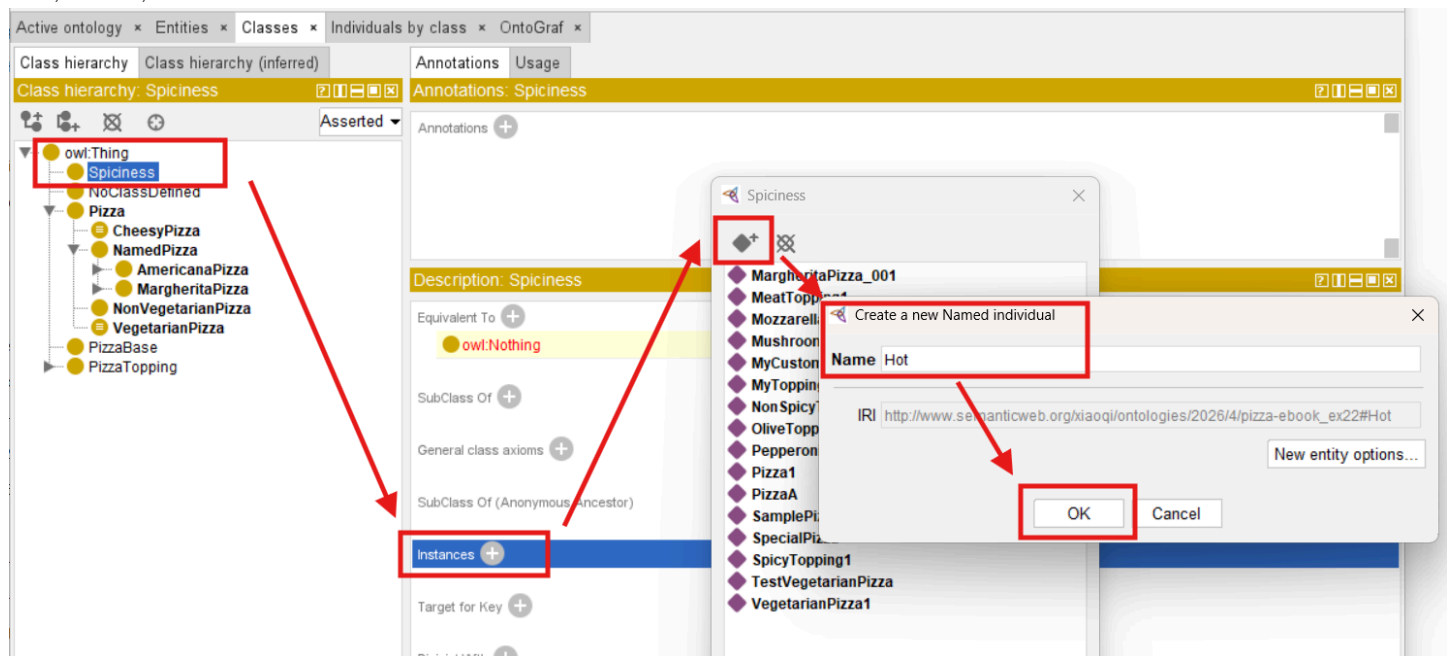
SubClass Of (Anonymous Ancestor)

Instances +

3. Click the Add icon (+) next to **Instances**



4. A dialog will appear with a diamond icon (create new individual) and a delete icon. Use the diamond icon to create three individuals in sequence: Hot, Medium, and Mild.



5. After creating, your 'Spiciness' class' individuals should be like below:

Active ontology x Entities x **Classes** x Individuals by class x OntoGraf x

Class hierarchy Class hierarchy (inferred) Annotations Usage

Class hierarchy: Spiciness

Annotations: Spiciness

Annotations +

Description: Spiciness

Equivalent To +

owl:Nothing

SubClass Of +

General class axioms +

SubClass Of (Anonymous Ancestor)

Instances +

Mild



Tip

You may also use **Individuals by class** tab to create those individuals, below is the screen of the three individuals after creating:

Active ontology x Entities x **Classes** x **Individuals by class** x OntoGraf x

Class hierarchy: Spiciness

Annotations Usage

Annotations: Mild

Annotations +

Description: Mild

Types +

Spiciness

Same Individual As +

Different Individuals +

Direct instances: Mild

For: Spiciness

Mild

Choose either way which you are preferred.

Note

You may notice that only one of the three new individuals is actually created as an instance of `Spiciness` at this point. That is expected. The next step will supply the reasoner with enough information to recognize all three as instances of the class.

Step 4: Define `Spiciness` as an enumerated class

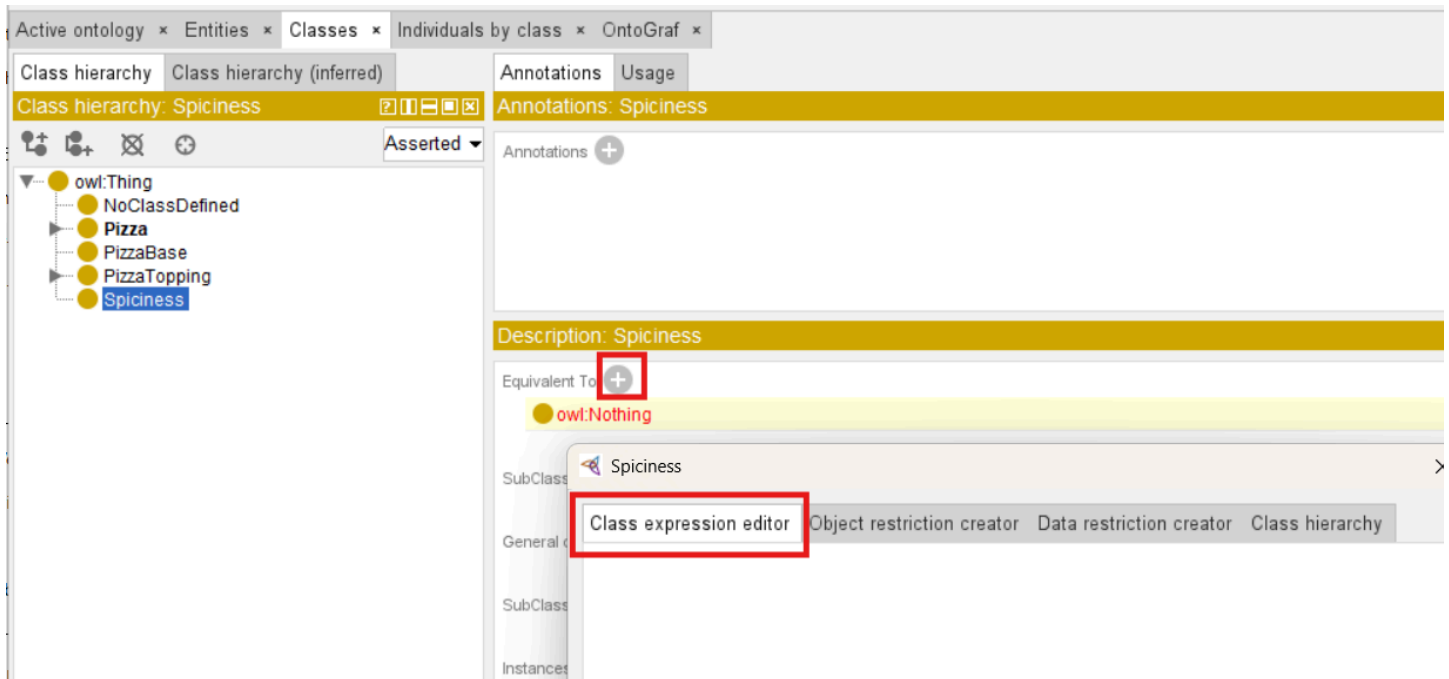
1. Make sure `Spiciness` is still selected

The screenshot shows the OntoGraf interface with the 'Classes' tab selected. The 'Class hierarchy' view on the left shows a tree structure starting from `owl:Thing`, with `Spiciness` highlighted. The 'Description' view on the right shows the 'Equivalent To' section with `owl:Nothing` listed.

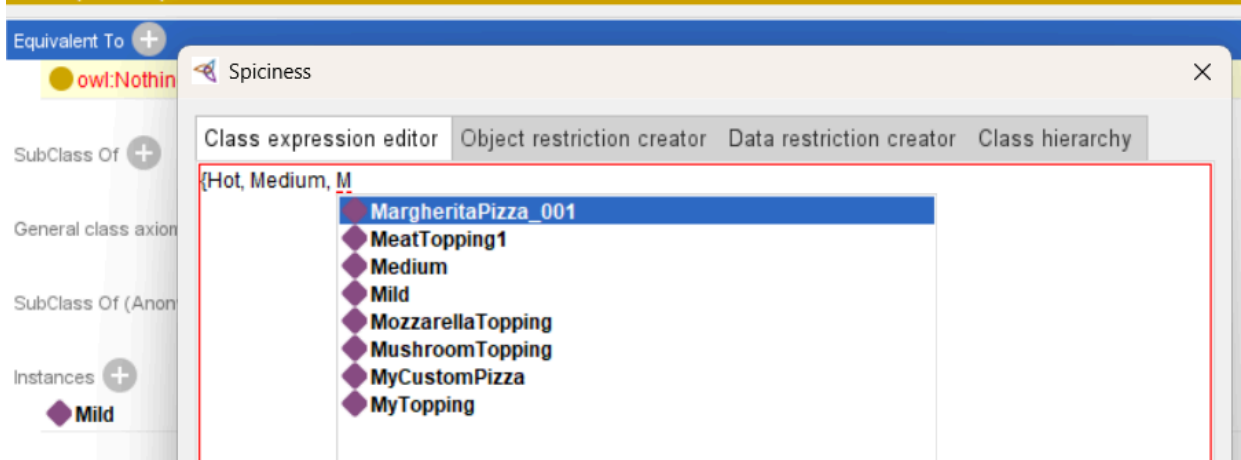
2. In the Description view, locate the **Equivalent To** section.

The screenshot shows the OntoGraf interface with the 'Classes' tab selected. The 'Description' view on the right shows the 'Equivalent To' section highlighted, with `owl:Nothing` listed.

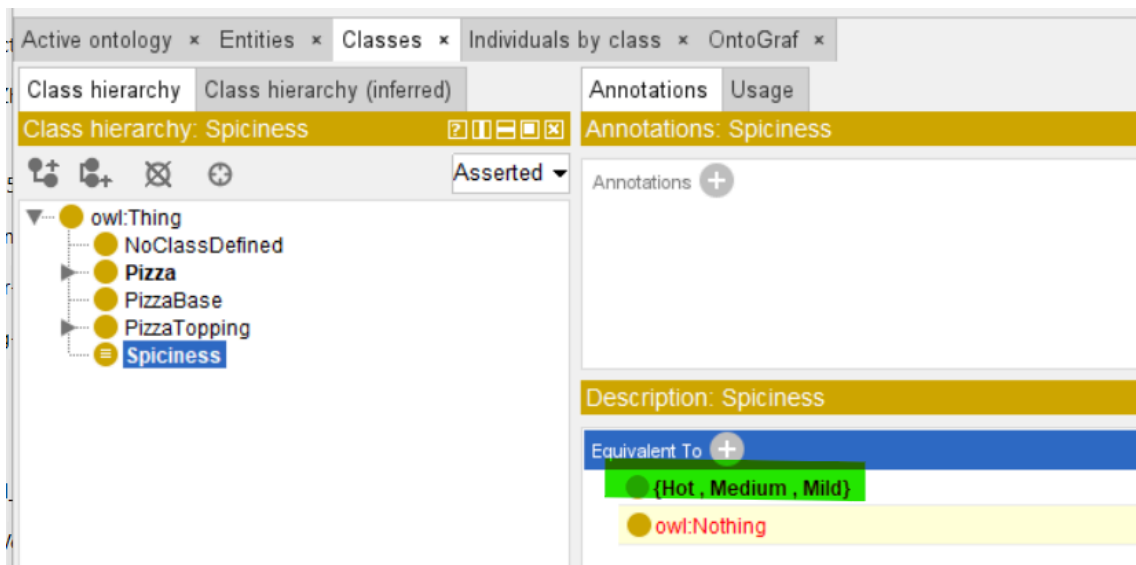
3. Click the Add icon (+)



4. In the Class Expression Editor, enter: {Hot, Medium, Mild} . (Tip: remember you may click Tab while typing to get possible choices)



5. Click ok .

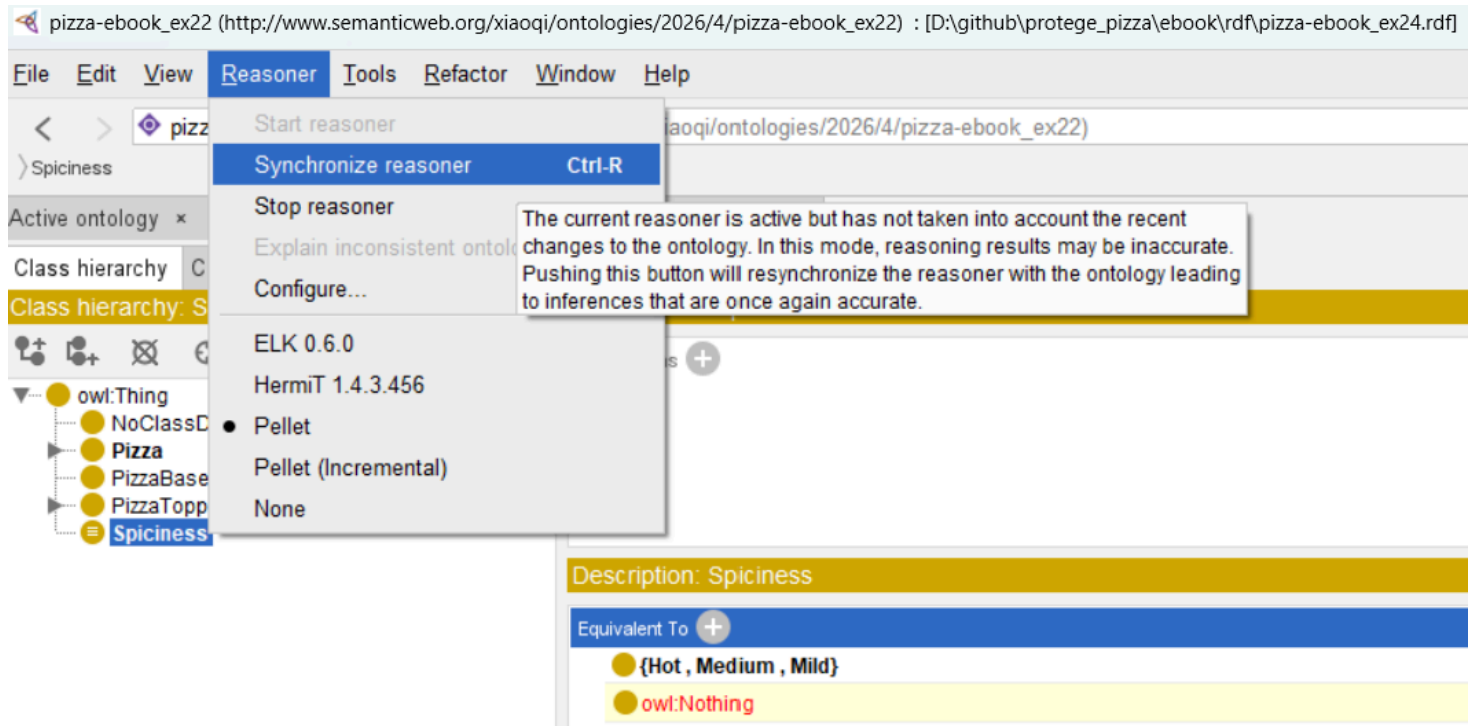


After this step, `Spiciness` class has the following description:

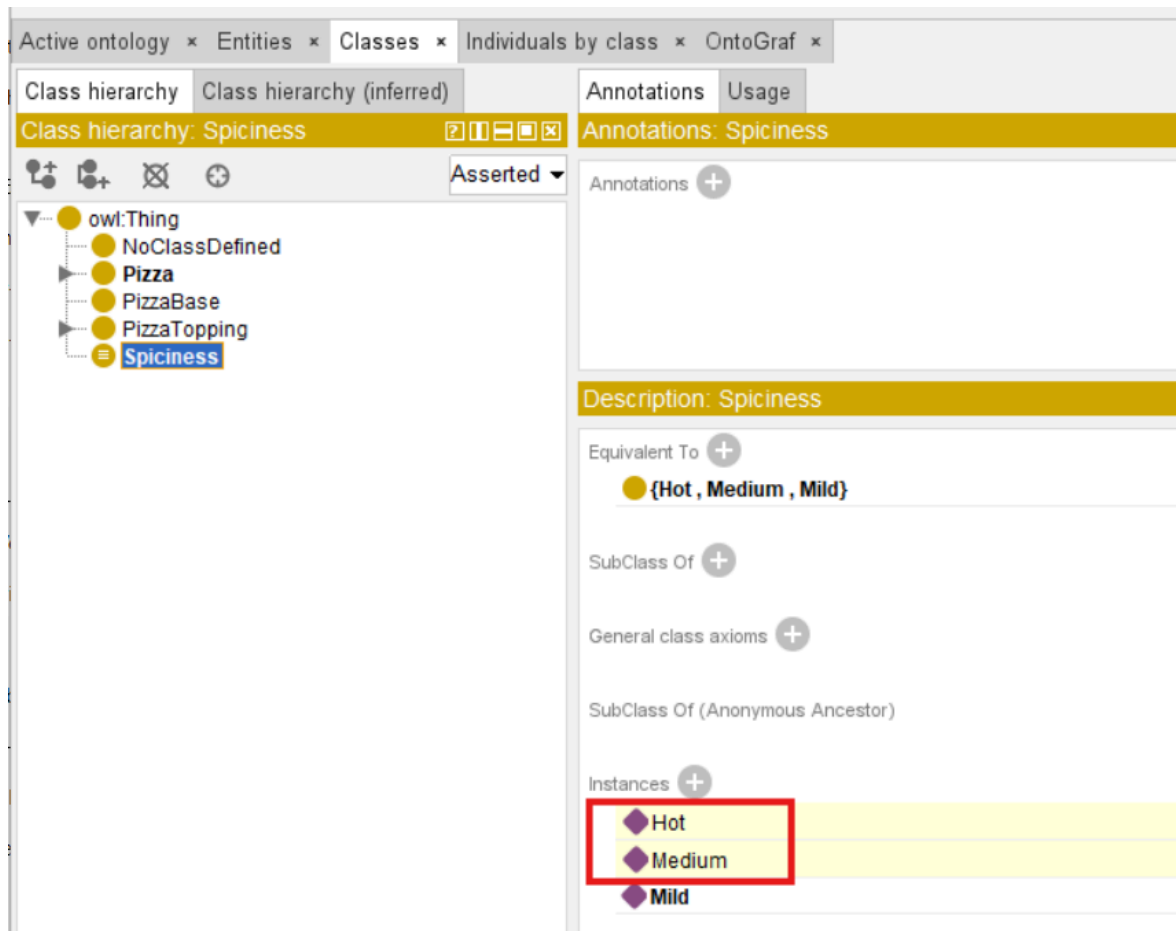
```
Class: Spiciness
  EquivalentTo: {Hot, Medium, Mild}
```

Step 5: Synchronize the reasoner

Select Reasoner > Synchronize reasoner (Ctrl+R)

**Step 6: Observe the result**

Look at Spiciness in the Classes tab.



The reasoner now recognizes `Spiciness` as a **defined class** (three horizontal stripes in the class icon), with exactly three instances: `Hot` , `Medium` , and `Mild` . No other individual can be an instance of `Spiciness` , because the class definition enumerates the complete set of allowed values.

The reasoner has done two things:

1. **Classified the three individuals as instances of `Spiciness`** -- even though we asserted only one of them directly, the enumeration definition now covers all three.
2. **Closed the value domain** -- the class `Spiciness` now has exactly three instances, and no more are possible.

This is the first time in the chapter that we have closed the world for *values* rather than for *properties*. The effect is immediate: what was previously an open-ended class with unknown membership is now a fully specified enumeration.

Key Insight:

"Before the `EquivalentTo` axiom, `Spiciness` was an empty class -- a name without an extension. After the axiom, `Spiciness` has a complete, closed-world specification: exactly three values, and no others."

24.8.3 The Result: A Deterministic Value Domain

The enumerated class has provided a **deterministic value domain** for `Spiciness` .

For execution, this is essential. When an execution system encounters a `Spiciness` value, it knows the value must be one of `Hot` , `Medium` , or `Mild` . It can handle all possible cases exhaustively.

Contrast this with the alternative. If we had only declared `Spiciness` as a class (with `Hot` , `Medium` , and `Mild` as asserted instances), the reasoner would still recognize those three individuals as instances of `Spiciness` . But it would not know that they are the *only* instances. Under the Open World Assumption, there might be other `Spiciness` values not yet asserted.

Enumeration removes this uncertainty. The `EquivalentTo {Hot, Medium, Mild}` axiom does two things simultaneously:

- It says that `Hot` , `Medium` , and `Mild` **are** instances of `Spiciness` (sufficient condition).
- It also says that anything else is **not** an instance of `Spiciness` (necessary condition).

This is what makes enumeration a closed-world specification.

Key Insight:

"Enumerated classes are the OWL equivalent of an `enum` in a programming language. They define the complete set of possible values -- nothing more, nothing less."

24.8.4 Why Enumerated Classes Matter for Execution

Enumerated classes are essential for execution because:

1. **The define finite value domains.** Execution systems need to know the complete range of possible values to handle every case.
2. **They enable exhaustiveness checking.** If an execution system handles `Hot` , `Medium` , and `Mild` , it has handled all cases -- because the ontology guarantees there are no others.
3. **They convert open-world value domains into closed-world specifications.** Without enumeration, a property might have unknown or additional values. With enumeration, all values are known.
4. **They compose with other execution primitives.** Enumeration works with `hasValue` restrictions (**Exercise 25**) and cardinality restrictions (**Exercise 26**) to build richer execution logic. For example, an execution rule can fire on `hasSpiciness` value `Hot` , confident that `Hot` is a fixed, well-known value from a closed enumeration.

The table below contrasts the behavior of a property with and without an enumerated value domain:

Aspect	Without Enumeration	With Enumeration
Value domain	Open -- any value is possible in principle	Closed -- only the listed values are possible
Reasoner behavior	Cannot exhaustively check value conditions	Can exhaustively check value conditions
Semantic strength	Description of a class	Specification of a value domain

Aspect	Without Enumeration	With Enumeration
Execution readiness	Not executable for value-based triggers	Executable for value-based triggers

Key Insight:

"Execution cannot handle unknown values. Enumeration is how the ontology guarantees that all values are known."

24.8.5 Engineering Perspective: Exercise 24 as a Case Study

From the perspective of Protégé, **Exercise 24** involves creating a class, creating three individuals, and adding a single `EquivalentTo` axiom. It appears to be a small modeling activity.

From the perspective of ontology engineering, however, something significant has occurred. The ontology now contains a **completely specified value domain** -- a closed-world set of possible values.

What was previously an open class has become a fully enumerated one.

The engineering contribution of **Exercise 24** can be summarized as follows:

Engineering Perspective	Contribution from Exercise 24
SKDL Stage	Stage 7 -- Executable Knowledge
Primary Objective	Introduce the enumerated class as a value-domain specification
Modeling Activity	Define <code>Spiciness</code> as <code>{Hot, Medium, Mild}</code>
Engineering Outcome	Provide a finite, deterministic value domain for execution; the reasoner now treats <code>Spiciness</code> as a closed-world enumeration
Preparation for Later Sections	Establish the enumerated value foundation for the <code>hasValue</code> restrictions in Exercise 25 and the cardinality-based logic in Exercise 26

Key Insight:

"**Exercise 24** is the moment the ontology specifies a complete value domain. This is essential for exhaustive execution logic."



Following the Tutorial

Exercise 24 in the original `Pizza` Tutorial introduces the enumerated class `Spiciness` with the individuals `Hot`, `Medium`, and `Mild`, and defines `Spiciness` as `EquivalentTo {Hot, Medium, Mild}`. The tutorial then demonstrates the reasoner recognizing all three individuals as instances of the class.

In this book, we have followed the tutorial exactly -- same class, same individuals, same axiom. The only difference is presentational: we have added the observation that this constitutes a *closed-world specification of a value domain*, which is what makes enumeration as execution primitive for **Stage 7**. The tutorial introduces the construct; the book frames it in terms of executability.